



A Test-Driven Approach for Refining Use Case Specifications of Software Requirements with LLMs

Haibo Li^(✉), Lixiao Zheng, and Qihang Cai

College of Computer Science and Technology, Huaqiao University, Xiamen 361021, China
{lihaibo, zhenglx}@hqu.edu.cn, 24014083004@stu.hqu.edu.cn

Abstract. Requirements described in natural language often require manual organization to align with specifications for test case generation. Similarly, when test cases are modified or enhanced, corresponding updates to the requirements must be performed manually to maintain consistency across design and development. UML use case specifications use textual descriptions to define scenarios and behaviors that achieve user goals. However, existing practices heavily rely on manual efforts to create and maintain use case specifications, leading to inefficiencies and potential inconsistencies. To address these issues, we propose a test-driven approach to further refine UML use case specifications, leveraging large language models (LLMs) to facilitate the synchronization of natural language requirements, UML specifications, and test cases. We introduce a structured format that enables LLMs to transform ambiguous natural language requirements into precise use case specifications compliant with UML, from which test cases can be automatically generated. Conversely, modifications to test cases are propagated back to update the corresponding use cases, ensuring bidirectional consistency. This iterative feedback loop continuously refines use case specifications, enhancing their accuracy and alignment with evolving requirements. Experimental evaluation on two real-world software projects demonstrates the practical application of our approach.

Keywords: software requirement · specification · UML · use case · test case · LLM

1 Introduction

Automatically generating test cases from software requirements is an important task in software engineering [1]. Acceptance testing is a critical phase for ensuring that the software system, as a whole, meets its functional requirements and business objectives, and for determining its readiness for delivery [2]. Effective acceptance testing requires well-defined test criteria, which help test engineers understand what to test and how to conduct tests. However, such criteria depend on clear user requirements. Natural language, with its inherent ambiguities, often introduces interpretational biases when describing requirements, making it impractical to directly derive test cases from such descriptions.

Existing manual methods—including the Given/When/Then structure, rule-based criteria, and scenario-based approaches—are still widely used but are labor-intensive and prone to inconsistencies.

To address these issues, researchers have turned to software requirements modeling techniques [3, 4] such as Unified Modeling Language (UML). UML use case modeling combines visual diagrams with detailed textual specifications, offering a comprehensive view of requirements. Use case specifications, in particular, include a main success scenario and alternative/exception paths. Despite their strengths, the descriptions of use case specifications are only guided at the principle level, including the use of simple logic and freedom from design-oriented interactions. The means of describing them remains natural language. This gap underscores the need for better ways to bridge requirements engineering and acceptance testing.

Recent advancements in large language models (LLMs) offer promising opportunities to address these weaknesses in requirements engineering [5, 6]. LLMs, known for their ability to process and generate human-like text, can be fine-tuned through iterative prompting to perform complex reasoning tasks [7]. For example, researchers have explored using LLMs to generate acceptance criteria directly from user stories [8]. While these efforts are promising, providing a comprehensive solution for requirements characterization and testing as a whole remains a challenge. Moreover, most existing studies use LLMs only for natural language processing, limiting their ability to tackle the deeper structural and semantic challenges of defining test cases [9].

To address these challenges, we propose a test-driven approach leveraging LLMs. This approach systematically formalizes and iteratively refines UML textual use case specifications through automated test generation and validation. We present a structured format that formalizes the description of both basic and alternative/exception within use case specifications. This structured representation enables the automatic generation of comprehensive test cases, ensuring full-coverage testing for specific requirements. Furthermore, LLMs assist in identifying and resolving missing or ambiguous elements by generating extended use cases. These extensions are then seamlessly integrated back into the refined requirements. Although UML use case modeling serves as the illustrative framework in this study, the core methodology is adaptable to other requirements modeling techniques.

By combining normalization techniques, test-driven development principles, and the reasoning capabilities of LLMs, this paper aims to provide a practical framework for enhancing the accuracy and completeness of software requirements while bridging the gap between use case specifications and acceptance testing. This approach not only improves the quality of test cases but also simplifies the construction of requirements, ultimately contributing to more robust and reliable software development. The primary contributions of this work are as follows:

- 1) **Use Case Specification Structure:** We propose a structure that formalizes the use case specification and supports the automatic generation of test cases that serve as full-coverage test scenarios for the specified requirements. This structure is designed to be understandable by LLMs, enabling effective utilization of their capabilities.
- 2) **Feedback-Enabled Refining Framework:** Our framework establishes a feedback loop between use case specification and generated test cases. Extended test cases, identified

and created with the help of LLMs as part of the test scenarios, are incorporated back into the requirements. This creates a closed-loop process that improves both requirements and testing outcomes.

The remainder of this paper is organized as follows: Sect. 2 reviews related work. Section 3 analyzes use case-based software requirements, test cases, and their mappings. Section 4 outlines our proposed framework for software requirements refinement and the specification structure. Section 5 details the evaluation results. Finally, Sect. 6 offers concluding remarks.

2 Related Work

The application of LLMs in software engineering has garnered significant attention in recent years [6, 7, 10, 11]. Models like OpenAI's GPT-4 have demonstrated remarkable capabilities in understanding and generating natural language, paving the way for automation and improvement in traditionally labor-intensive software requirements engineering (SRE) tasks [5, 6]. Accurate processing and analysis of software is critical to the success of any software project, and LLMs offer a promising avenue for addressing longstanding challenges in this domain.

Recent studies highlight the potential of LLMs to enhance several SRE processes. These include requirements model extraction [12–14], classification [15], ambiguity detection [16, 17], and refinement. For example, Fantechi et al. [18] showed that ChatGPT can detect inconsistencies in requirements with an accuracy comparable to human experts. Krishna et al. [19] demonstrated the ability of LLMs to automate the generation of software requirements specifications (SRS), significantly improving productivity and reducing development time. Lubos et al. [9] investigated how LLMs can refine requirements following ISO 29148 standards, thereby improving quality and standardization. Similarly, Ren et al. [20] proposed a framework for using LLMs to generate goal models from user reviews, enabling developers to identify additional requirements. Bertram et al. [21] leveraged NLP techniques to enhance the consistency of requirements specifications through LLM-based automation, while Hasso et al. [22] demonstrated that dynamic question generation using LLMs can improve precision and completeness. Luitel et al. [23] further investigated how BERT can identify incomplete natural language requirements by integrating contextual and machine learning-based analysis. Moreover, complementary approaches utilizing LLMs to analyze requirements consistency and detect incompleteness have shown their potential to simplify requirements analysis [24].

In parallel, the standardization of requirements to improve quality and consistency has become an important area of focus. Yang et al. [25] proposed automating the generation of prototypes from formal requirements models for validation. Other studies introduced template structures for normalizing or semi-formalizing requirements to address inconsistencies and ambiguities [16, 26]. Some approaches used LLMs to extract semantic relationships from requirements to enhance consistency analysis [14], while others transformed vague user requirements into structured data that can be mapped to application services [15]. Although the idea of structured requirements has been explored for years, their practical adoption has been limited due to challenges in implementation and integration.

Another closely related domain examines the mapping of software requirements to test cases to ensure traceability and comprehensive testing [27]. Generating test cases directly from requirements has been shown to improve coverage and traceability, but it remains underexplored in SRE and software engineering due to the complexity of handling ambiguous natural language requirements. Mustafa et al. [1] highlighted this challenge, emphasizing the barriers created by incomplete and vague requirements. To address the problem, Jiao et al. [28] developed a framework for parallel test case generation using Natural Language Processing (NLP), while Wang et al. [29] proposed a method to automatically generate acceptance test cases directly from use case specifications. These efforts significantly reduce manual effort while improving test coverage. Alternative methods, such as leveraging mutation testing to generate test cases [30] or using tools designed for safety-critical systems to extract clear, unambiguous requirements for test generation [2], have also been explored. Kiyoshi et al. [31] advanced automated test case generation using topic modeling to match similar requirement specifications, reducing training sample requirements and enhancing accuracy.

Despite these advancements, notable limitations remain. Existing studies have primarily focused on high-level requirements, while paying insufficient attention to ensuring consistency between use cases and test cases in terms of the more specific low-level requirements.

3 Requirement Specification and Test Case Analysis

3.1 Software Requirement Description in Use Case

High-level requirements typically describe the overall goals, core functions, business value, and scope boundaries of a system. For example, “the system should allow registered users to purchase goods online”. In contrast, low-level requirements elaborate on specific functional points, behaviors, rules, constraints, and acceptance criteria necessary to fulfill high-level requirements. For example, “users must log in before they can add goods to the shopping cart”. One of the core activities in requirement analysis is to gradually refine and decompose high-level requirements into low-level ones.

Use case modeling refers to the process of describing the functional requirements of a system using use cases. A use case model mainly consists of two components: use case diagram and use case specification. Among them, the use case specification contains both business goals and macro-oriented business steps, as well as business rules and interactions rules that are gradually refined to low-level details. It carries a complete requirement chain from high-level goals to low-level details. The acceptance criteria implied in the process steps can be directly transformed into test cases. Therefore, the use case specification is crucial for software acceptance testing.

A use case specification contains a detailed description of a use case and usually conforms to a template [32]. A use case consists of a basic flow of events, which describes the scenario in which the use case can be successfully executed; alternative flows, which represent optional steps within the use case—typically triggered by specific conditions and branching off from the basic flow; and exception flows, which outline steps for handling potential conditions that would prevent the use case from succeeding. For

example, a use case for handling telephone orders from buyers can be described, as shown in Fig. 1.

Use Case ID	01
Intent	Company handles phone orders of buyers
Role	order clerk
Pre-conditions	
Post-conditions	buyer completes payment.
Basic Flow	
1. Buyer calls in with a purchase request. 2. Company captures buyer's name, address, requested goods, etc. 3. Company gives buyer information on goods, prices, delivery dates, etc. 4. Buyer pays for order. 5. Company creates order, ships order to buyer. 6. Company ships invoice to buyer. 7. Buyer pays invoice.	
Alternative Flow(s):	
3a. Company is out of one of the ordered items: 3a1. Renegotiate order. 4a. Buyer pays directly with credit card: 4a1. Take payment by credit card (use case 44) 7a. Buyer returns goods: 7a1. Handle returned goods (use case 105)	
Exception Flow(s):	
4a. User payment fails 4a1. The system prompts the user to change the payment method.	

Fig. 1. An example of textual use case template

3.2 Mapping Between Use Case Specifications and Test Cases

There is a mapping relationship between use case specifications and test cases, analyzed primarily from software engineering principles.

In classical software engineering, which follows a top-down development approach (such as the traditional V-model and its variants like double or triple V-models [33]), acceptance testing has a special role: it directly validates alignment with use case specifications, ensuring the software meets user expectations and business needs. This makes it a critical check to decide if the software is ready for delivery.

In agile development, the relationship evolves dynamically. Use case specifications are often simplified into user stories, with acceptance criteria embedded within them. Here, acceptance tests are co-created by developers, testers, and stakeholders (e.g., via Behavior-Driven Development) and act as living documentation. They not only verify compliance with the evolving use case details but also drive development increments, with test cases refining and clarifying use case specifications iteratively.

Test scenarios, derived from use case flows (basic and alternative), ensure requirement coverage, with mapping rules facilitating systematic translation from flows to test cases.

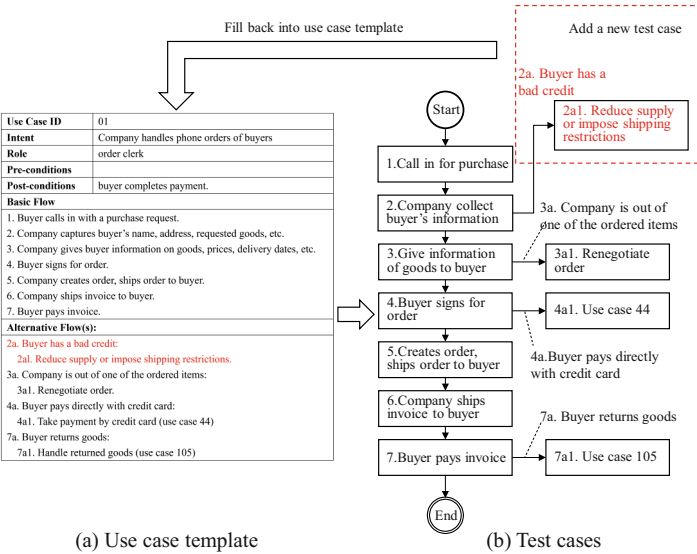


Fig. 2. An example of mapping between a use case and the test cases

The process for generating test cases from use case specifications can be outlined as follows:

- 1) Begin with the basic flow.
- 2) Extend the basic flow with individual alternative steps that branch from it, ensuring each step in the alternative flows is used at least once and leads to the use case's conclusion.

The basic flow, as shown in Fig. 2(a), can generate a test case, while the basic flow combined with an alternative step can produce a new test case. Test cases generated from this process might include: 1) the basic flow, 2) the basic flow with alternative step 3a1, 3) the basic flow with alternative step 4a1, and 4) the basic flow with alternative step 7a1, as shown in Fig. 2(b). If additional test cases, such as step 2a1, are required, they can also be directly mapped to specific use case flows.

4 The Refinement Framework

4.1 An Overview

To support full-coverage business scenarios essential for system or acceptance testing, this paper proposes a framework for refining use case specifications, as illustrated in Fig. 3. The framework adopts a predefined structure which guides LLMs to translate natural language requirements into structured UML use case specifications. These

structured use case specifications can then be directly used to generate test cases based on mapping rules, which help test engineers grasp accurate and comprehensive business scenarios. To normalize this process, each step of both basic and alternative flows within use case specifications is described using a structured format. This ensures the requirements are suitable not only for automated test generation but also for other software engineering tasks, such as identifying class diagrams in object-oriented design.

Additionally, when new test cases are generated by the LLMs, the newly added alternative steps can be filled back into the textual use case specifications according to this structure. LLMs can enhance user requirements by introducing extended use cases, providing further refinement for requirements engineers.

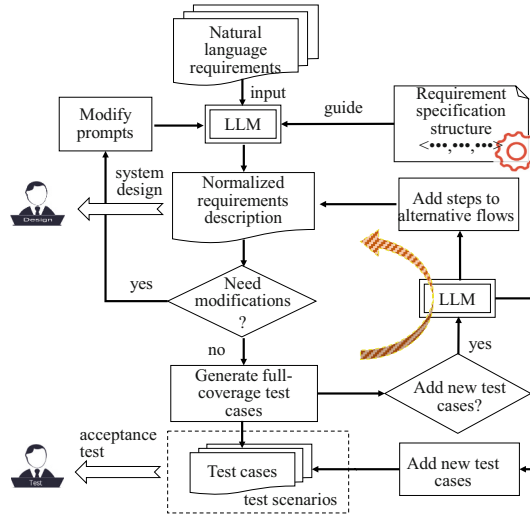


Fig. 3. The test-driven refinement framework

4.2 Requirement Specifications Structure

The UML use case template is too “coarse-grained”. If test cases are to be automatically generated from natural language requirements, the natural language still needs to be further structured. This is very important for generating test cases (which require machine-readable requirements) and facilitating object-oriented design (which requires clear identification of actors and affected business objects). For this purpose, we set up a structured text format for natural language requirements, where each sentence follows the format below:

$$< \textit{Pre - conditions}[\textit{Previous Step}], \textit{actor}, \textit{action}, \textit{business objects}, [\textit{toactor}], [\textit{Post - conditions}] > \quad (1)$$

This structure uses angle brackets to enclose the requirement’s core description. The element “*Pre-conditions*” specifies prerequisites, which may also be interpreted as preceding steps. “*Business object*” denotes a real-world entity (e.g., order, invoice) impacted by the “actor” (the entity initiating the action). “*Post-conditions*” define the

conditions that must be held after successful execution. For example, the natural language requirement “*Buyer creates order, Company ships order to the buyer.*” can be expressed as the structured format $\langle \textit{Buyer, creates, order} \rangle$ and $\langle \textit{if buyer creates order, Company, ships, order, to buyer} \rangle$.

This standardized structure ensures requirements are both machine-readable and human-understandable, facilitating automated processing by LLMs. If other use cases need to be executed in alternative flows, the requirement specification structure can be simplified to the following form:

$$\langle \textit{Pre – conditions [Previous Step], use case ID} \rangle \quad (2)$$

For example, the natural language requirement “*buyer returns goods, handling returned goods.*” can be expressed as “*if buyer returns goods, use case 105*”, because handling returned goods is a separate use case.

4.3 Refining Requirement Using LLMs

Integrating LLMs into this framework is critical for refining requirements. This process involves systematically transforming raw, natural language requirements into structured descriptions, generating UML use case specifications, and creating high-quality test cases. Furthermore, an iterative feedback mechanism makes it possible to refine the use case specifications and improve subsequent outputs. Below, the components of the proposed approach are outlined in detail.

1) Structured Use Case Specifications

The first step in our framework is converting raw natural language requirements into the structured format. Contextual knowledge relevant to software requirements, including the elements of the specified structure (e.g., pre-conditions, actor, action, post-conditions, etc.) is provided to LLMs to ensure they fully understand the task. LLMs are guided with the following prompt:

“Given the following unstructured software requirement described in natural language, structuring it according to the predefined requirements pattern. Ensure each sentence follows the $\langle \textit{Pre-conditions, actor, action, business objects, to actor, Post-conditions} \rangle$ pattern and includes pre-conditions, post-conditions, and exceptions (if any). Present the structured requirement in a clear and concise format.”

Once structured, the requirements are further processed to generate UML use case specifications. Using this prompt:

“Based on the following structured requirement, create a UML use case specification that includes intent, role, preconditions, post-conditions, and basic/alternative flows. Ensure the description is clear and comprehensive.”

The output provides well-structured, detailed use case specifications that are optimized for accuracy and readability. These specifications can be further refined based on

new requirements or feedback from testing phases, ensuring they meet the precise needs of various stakeholders.

2) Automatic Test Case Generation

To generate test cases from the UML use case specifications, we leverage LLMs with targeted prompt:

“Given the UML use case: [insert UML use case template here], generate a set of test cases that cover both the basic and alternative flows. Each test case should detail the test data, steps, and expected results with clarity and precision.”

This ensures that the generated test cases comprehensively cover both intended functionality and edge cases derived from alternative flows. Additional refinement can be introduced by specifying the desired test case format or including examples of well-formed test cases to guide the LLM’s output.

3) Iterative Refinement of Requirements Based on Test Case Reviews

Test case reviews help find flaws in original requirements, such as gaps, ambiguities or inconsistencies. These flaws are then fixed to improve test scenarios and requirement specifications. For example, uncovering unconsidered exceptions may lead to new requirements, while mismatches between requirements and test cases prompt revisions to align them. This iterative feedback keeps requirements in step with evolving system understanding. LLMs can aid this review via prompts:

- Identifying Gaps and Inconsistencies

“Analyze the following test cases to check if there are any contradictions within the test steps, if any steps are missing, or if the expected results are not clearly defined.”

- Suggesting Additional Test Cases

“Based on the following test cases, suggest additional test cases that would provide more comprehensive coverage of the functionality being tested, such as edge cases, boundary conditions, or error handling.”

- Discovering Implicit Requirements

“Analyze the following test cases and identify underlying assumptions about the system’s behavior that may not be explicitly documented in the requirements.”

Feedback from these analyses is then used to refine the requirements further. For example: *“Based on the feedback from the test case review: [insert feedback], refine the following requirement: [insert requirement]. Ensure that the refined requirement addresses the identified issues”*. Additional constraints, such as ensuring the refined requirement aligns with SMART principles (Specific, Measurable, Achievable, Relevant,

and Time-bound), can also be incorporated. Iterative refinement not only improves the quality of requirements but also enhances the LLM’s accuracy in subsequent tasks as it learns from prior feedback.

This closed-loop process integrates LLMs to streamline requirement refinement and test case generation, ensuring that requirements evolve iteratively while maintaining alignment with testing efforts. By automating these processes, the framework reduces ambiguity and enhances the precision of software requirements, enabling systems or acceptance testing in full-coverage test scenarios for the specified requirements.

5 Evaluation

5.1 Set up

To evaluate our proposed approach, we selected software requirements from the PURE dataset [34], a publicly accessible repository of natural language requirements documents sourced from various online platforms. We identified two specific projects from the dataset based on following selection criterion: their documentations contain detailed natural language descriptions of software functionalities, providing rich semantic material for use case extraction. The details of these two projects are as follows.

- 1) KeePass password management system is a lightweight application designed to address the challenge of securely managing and storing passwords. Its primary function is to maintain a secure, encrypted database where users can store passwords, usernames, email accounts, and related information, all protected by a master password.
- 2) GAMMA-J Web Store is an online E-commerce system designed to allow new online store owners a quick and easy means to setup and perform sales and other core business over the internet. Its primary function is to manage customer accounts, manage customers’ shopping carts and confirm orders, manage online store inventory, and so on.

Since the PURE dataset also includes non-functional details such as implementation and technical aspects, we filtered the data, extracting only functional requirements to be used as input for structuring in our framework. To better validate our approach, we also extended natural language description for some of the requirements when necessary.

GPT-4, a LLM developed by OpenAI, was selected for this experiment due to its ability to process and generate human-level text. The experimental process adopted a “human-in-the-loop” approach, where human reviewers and the language model collaborated iteratively. Key steps in the process included structuring natural language requirements, transforming natural languages into structured use case specification, generating test cases, reviewing test cases, and refining use cases based on feedback. Supplemental project information, such as scope and an overview of the KeePass system or Web Store system, were also provided to the LLM to help it generate more accurate and relevant outputs during the iterative process.

Our experimental evaluation¹ consists of two parts. First, as a demonstration example, one of the authors created use cases for the KeePass requirements by following our

¹ Data available at <https://github.com/HQU-CaiQihang/data>.

proposed method. Second, for comparative analysis, we conducted a study to evaluate the performance of two groups of requirements engineers in analyzing requirements for all the functional descriptions of the Web Store system.

5.2 Example on KeePass

Below, we outline each step following our approach using an example extracted from the KeePass requirements document. One of the natural language requirements in KeePass describes a feature for opening an existing database. The original description is stated as follows:

“When choosing to open a database a user is transferred to his documents where he navigates to find the database he wants. When the database is found, the master password is wanted so that the database will be unlocked. Once this is done the user is free to access his data. The user can view, modify, add, or delete the data if he has the proper permissions. The user also can perform backup and recovery operations on the data. If the user closes the database, the system returns to the navigation interface of the document.”

The refinement follows three key steps:

1) Natural Language Structuring

The original natural language requirement is structured into a unified format as described in Sect. 4.2. The LLM-generated description (with minor manual refinements) is presented in Table 1.

Table 1. The structured requirement description

No.	The Structured Natural Language Requirement
1	<[User initiates database opening process], User, navigates, database>
2	<[User navigates to database], System, prompts for, master password>
3	<[System prompts for master password], User, enters, master password>
4	<[User enters master password], System, verifies, master password>
5	<[Password is correct], System, unlocks, database>
6	<[Password is incorrect], System, displays, error message>
7	<[System unlocks database], User, accesses, database content>
8	<[User has the proper permissions], User, view/modify/add/delete, data>
9	<[User has the proper permissions], User, performs, backup and recovery operations>
10	<[User closes database], System, returns, navigation interface of the document>

In addition, Deepseek is chosen as a baseline LLM, and one version of the generated structured requirement descriptions is shown in Table 2. Through comparison, this version strictly complies with the original requirement descriptions. The capability

of LLMs largely depends on prompt engineering, and various LLM products are also constantly evolving. In the future, requirement descriptions can be organized through more complex structures.

Table 2. The structured requirement description by Deepseek

No.	The Structured Natural Language Requirement
1	<i><[User chooses to open a database], User, Navigates through documents, Database></i>
2	<i><[Database is found], User, Enters, Master password></i>
3	<i><[Database is unlocked], User, Accesses, Data></i>
4	<i><[Database is unlocked + user has permissions], User, Views, modifies, adds, or deletes, Data></i>
5	<i><[Database is unlocked], User, Performs backup/recovery operations, Data></i>
6	<i><[User finishes data interactions], User, Closes database, Database></i>
7	<i><[Database is closed, System], Returns to, Navigation interface></i>

2) Use Case Template Transformation

Since the structured requirements align with the use case specification format, the transformation process requires only minimal adjustments. The structured requirements are easily adapted into a UML use case template using either a manual process or the LLM. The resulting use case includes eight steps in the basic flow and four steps in alternative flows (steps 3a1, 4a1, 6a1, and 6b1). Figure 4 shows a detailed view of the basic and alternative flows.

Use Case ID	03
Intent	User accesses database content
Role	End users/Desktop
Pre-conditions	
Post-conditions	
Basic Flow	
1. User opens KeePass and selects "Open Database" operation. 2. System prompts for master password. 3. User enters master password. 4. System verifies master password. 5. System unlocks database. 6. User access his data: views, modifies, adds, or deletes the data, or perform backup and recovery operations on the data. 7. User closes database. 8. System returns to the main window.	
Alternative Flow(s):	
1a. User selects a type of folder non suitable for database. 1a1. System displays "file not found" message. 1b. There are too much databases in the system. 1b1. System provides a database search function. 3a. The system checks user has no permissions to access the database. 3a1. The system displays error message and access is denied. 3a2. User resets the master password. 4a. System verifies master password and password is incorrect. 4a1. System displays error message. 6a. User forgets to save modification. 6a1. The system saves user modifications in real time. 6b. User operations are inactive for long periods of time. 6b1. System automatically locks the database and the master password to be entered again for continued access.	

Additional steps

Fig. 4. The flows in the use case template of KeePass example

3) Test Case Review and Iterative Refinement.

The generated test cases, as shown in Table 3, are then reviewed by test engineers or by LLM. In our experiments, we tasked GPT-4 with reviewing the generated test cases, and the missing test cases are proposed, as shown in Table 4.

Table 3. The flows in the use case specifications generating test cases

No.	The Flows in the Use Case that Generate Test Cases
1	Basic flow
2	Basic flow + step 3a1 in alternative flow
3	Basic flow + step 4a1 in alternative flow
4	Basic flow + step 6a1 in alternative flow
5	Basic flow + step 6b1 in alternative flow

Table 4. The added test cases and flows in use case specification

No.	Test Cases	Flows in Use Case	Supplemental Functionality Requirement
1	<i>“If User selects a type of folder non suitable for database, System displays “file not found” message.”</i>	Basic flow + step 1a1 in alternative flow	Database navigates functionality: <i>“When User selects a type of folder non suitable for database, the system displays “file not found” message.”</i>
2	<i>“User can search the database if it is difficult to find.”</i>	Basic flow + step 1b1 in alternative flow	Database search functionality: <i>“System provides a database search mechanism if there are too many databases in the navigation page, so that the user can easily find the database he wants.”</i>
3	<i>“user resets password if forgotten.”</i>	Basic flow + step 3a2 in alternative flow	Password reset functionality: <i>“System provides a password reset mechanism such that the user is able to successfully reset the master password and regain access to the database.”</i>

Its feedback includes suggestions for improvements and additional flows in use case specification. Using this feedback, the functional requirements are refined further by adding a new feature, as shown in Table 4. The corresponding steps are 1a1, 1b1, and 3a2, as shown in Fig. 4.

We compared other test case generation tools, such as TestSonic. The test cases generated by these tools with AI functions are comparable to the method in this paper in terms of quantity and quality, but they cannot feed back to the original requirements. Those without AI functions are basically unable to expand and supplement test cases from the original requirement descriptions.

This iterative process of interaction with LLM continues until the requirement description sufficiently meets the needs of all stakeholders. Each iteration improves the clarity and completeness of the requirements while identifying and addressing potential gaps in test coverage.

5.3 Comparison Analysis

5.3.1 Effectiveness Comparison

We conducted a comparative study to evaluate the performance of two groups of requirements engineers in analyzing software requirements and developing test cases for all the

functional descriptions of Web Store. Each group consists of three engineers, differentiated by their level of experience: junior (1–2 years), mid-level (3–5 years), and senior (6–8 years). One group utilizes the LLM GPT-4 following our approach, while the other serves as a baseline by performing tasks manually without the test-driven approach or LLM assistance, that is, the traditional manual method of analyzing requirements. The study focuses on assessing two aspects: the effectiveness and efficiency of the approaches.

Table 5 summarizes the results of use case specifications for Web Store created by engineers of different experience levels of each group. The first row shows the use case specifications in the original Web Store document. The results show that the number of basic flows in the use case specifications remains consistent regardless of the use of LLM. Both groups identified fewer use cases than the dataset due to redundant use cases in the raw requirement. LLM assistance significantly increased alternative flow identification, particularly for mid-level and junior engineers. Senior engineers exhibited inherent proficiency in detecting edge cases, resulting in more comprehensive and accurate requirement specifications.

These findings highlight the value of LLM in augmenting human analytical capabilities, particularly in identifying exceptional cases and uncovering hidden requirements. Additionally, senior engineers display an inherent strength in identifying exceptional scenarios, resulting in more comprehensive specifications.

We evaluated the quality of the use case specifications according to the following four characteristics defined in ISO 29148 [35].

- 1) Complete: All information needed to understand the requirement is included in the description (completeness).
- 2) Correct: The need is accurately represented in the requirement (consistency).
- 3) Unambiguous: The requirement is clearly stated, understandable, and allows only one interpretation (accuracy).
- 4) Verifiable: The requirement is formulated in a way that its fulfillment can be proven or, in the best case, measured (testability).

Table 5. Use case specification results for Web Store.

Mode	Level	Number of		
		Use Cases	Basic Flows	Alternative Flows
Original	–	26	26	14
Our approach	Junior	20	20	36
	Mid-level	20	20	42
	Senior	20	20	32
Baseline manually	Junior	20	20	33
	Mid-level	21	20	14
	Senior	20	20	26

The specifications were reviewed and scored by five independent experts, three from academic institutions and two from industry, and each with more than 10 years of experience in requirements engineering. The experts evaluated the specifications using a scale of 1 to 10 for each criterion.

Table 6. Quality of use case specification for Web Store

Mode	Level	Completeness	Consistency	Accuracy	Testability
Original	–	6.1	6.7	5.8	7.1
Our approach	Junior	7.8	8.2	8.5	6.8
	Mid-level	8.9	8.8	9.1	8.3
	Senior	9.5	9.1	9.6	9.5
Baseline manually	Junior	5.2	6.3	5.4	6.7
	Mid-level	6.4	7.7	6.8	8.1
	Senior	8.4	8.9	8.6	9.4

As shown in Table 6, LLM-assisted use cases consistently outperform manual efforts across all quality dimensions. By structuring the raw requirements, the LLM facilitates the identification of both basic and alternative flows while also reducing ambiguities and incorrectness, thereby improving the accuracy and consistency of the specifications. Moreover, adopting a test-driven approach to iteratively enhance the use case specifications effectively improves their completeness and testability.

To further evaluate our approach, we compared three use case specifications for Web Store: 1) the original use case specification from the PURE dataset, 2) for benchmarking validation, without LLM assistance, a manually created specification developed by an expert-level engineer with over 15 years of experience reverse-engineered the functional requirements by analyzing the requirement documentation, and 3) the specifications generated by three engineers, junior, mid-level, and senior engineers using our test-driven LLM approach. Removing the 6 duplicate use cases, there are 20 available use case templates in Web Store. The reverse-engineered requirement specification developed by the expert-level engineer served as the ground truth for evaluating precision and recall across the three specifications, calculated using the following two formulas:

$$precision = \frac{\#flows\ matching\ the\ ground\ truth}{\#total\ flows\ in\ the\ specification} \quad (3)$$

$$recall = \frac{\#flows\ matching\ the\ ground\ truth}{\#total\ flows\ in\ the\ ground\ truth} \quad (4)$$

The evaluation was conducted by the two experts to assess alignment and discrepancies between the use case flows (basic and alternative flows) in each specification and the ground truth. The experts made their judgments independently and then cross-validated their assessments. The results of this comparison are summarized in Table 7. We analyze the results as follows.

Table 7. Comparison of different use case specifications for Web Store

Use Case Specifications	Precision	Recall
Original	44.82%	33.33%
Junior	50.77%	76.92%
Mid-level	52.63%	84.62%
Senior	65.38%	87.18%

- 1) Original use case specifications achieved a precision rate of 44.82% and a recall rate of 33.33%, indicating suboptimal performance where over half of identified cases were irrelevant and two-thirds of relevant cases were missed.
- 2) The specifications obtained following our approach achieved higher precision compared to the original specification. Notably, engineers leveraging LLM achieved a higher recall rate than precision. This demonstrates that our test-driven, LLM-assisted approach helped identify most additional alternative flows, which allows for a broader scenario coverage despite potential trade-offs in precision.

5.3.2 Efficiency Comparison

To assess efficiency, we measured the time taken by both groups to complete each functional requirement specification for Web Store. For the group using our approach, we also recorded the number of times participants refined their LLM prompts and refined the use case specifications. In the baseline manual group, the total time spent in constructing the use case specifications was recorded. The results are shown in Table 8.

Table 8. Efforts saved by using our approach.

Level	Refine Times	Overall Time consumed(min)			Satisfaction
		<i>Our approach</i>	<i>Baseline manually</i>	<i>Speed up</i>	
Junior	11	63	645	10x	high
Mid-level	9	38	313	8x	high
Senior	7	30	156	5x	high

The analysis shows that the use of our approach results in a significant 5–10 fold increase in productivity compared to manual approaches. The reduction in manual effort allows participants to complete their tasks significantly faster. Moreover, the group reports high satisfaction with the test-driven approach facilitated by LLM. Junior engineer requires more time and need timely adjustments compared to their mid-level and senior counterparts, highlighting the need for additional training in the use of the LLM and software requirements engineering fundamentals.

6 Conclusion

This paper explores the UML use case specification of software requirements and the mapping rules between use case specifications and test cases. We present a test-driven approach that employs LLMs to bridge the gap between natural language and structured descriptions. Our findings demonstrate that LLMs effectively identify and transform requirements into the proposed structure, enabling the partial or full generation of use case specifications and test cases. Our case study on KeePass evaluated this approach with requirements engineers of varying experience levels. The case study on Web Store demonstrated improvements in both the efficiency and effectiveness of requirements analysis. Specifically, the test-driven and LLM-enhanced method facilitated the identification of additional alternative flows and produced more structured, accurate, and testable requirements.

Future research will aim to further enrich requirements documentation to generate higher quality test cases by incorporating inputs from business process models developed during the design phase. Additionally, we plan to explore alternative perspectives to further refine and optimize software requirements.

Acknowledgments. This work is supported by the Fundamental Research Funds for the Central Universities (No. ZQN-1010), Fujian Province Science and Technology Plan, China (No. 2019H0017).

References

1. Mustafa, A., et al.: Automated test case generation from requirements: a systematic literature review. *Comput. Mater. Contin.* **67**(2), 1819–1833 (2021). <https://doi.org/10.32604/cmc.2021.014391>
2. Moitra, A., et al.: Automating requirements analysis and test case generation. *Requir. Eng.* **24**(3), 341–364 (2019). <https://doi.org/10.1007/s00766-019-00316-x>
3. Necula, S.C., Dumitriu, F., Greavu-Şerban, V.: A systematic literature review on using natural language processing in software requirements engineering. *Electron.* **13**(11) (2024). <https://doi.org/10.3390/electronics13112055>
4. Ayenew, H., Wagaw, M.: Software test case generation using natural language processing (NLP): a systematic literature review. *Artif. Intell. Evol.* 1–10 (2024). <https://doi.org/10.37256/aie.5120243220>
5. Arora, C., Grundy, J., Abdelrazek, M.: Advancing requirements engineering through generative AI: assessing the role of LLMs. *Gener. AI Eff. Softw. Dev.* 129–148 (2024). https://doi.org/10.1007/978-3-031-55642-5_6
6. Vogelsang, A.: From specifications to prompts: on the future of generative large language models in requirements engineering. *IEEE Softw.* **41**(5), 9–13 (2024). <https://doi.org/10.1109/MS.2024.3410712>
7. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural. Inf. Process. Syst.* **35**, 24824–24837 (2022)
8. Li, Y., Keung, J., Yang, Z., Ma, X., Zhang, J., Liu, S.: SimAC: simulating agile collaboration to generate acceptance criteria in user story elaboration. *Autom. Softw. Eng.* **31**(2) (2024). <https://doi.org/10.1007/s10515-024-00448-7>

9. Lubos, S., et al.: Leveraging LLMs for the quality assurance of software requirements. In: Proceedings of the IEEE International Conference on Requirement Engineering, pp. 389–397 (2024). <https://doi.org/10.1109/RE59067.2024.00046>
10. Norheim, J.J., Rebentisch, E., Xiao, D., Draeger, L., Kerbrat, A., De Weck, O.L.: Challenges in applying large language models to requirements engineering tasks. *Des. Sci.* **10** (2024). <https://doi.org/10.1017/dsj.2024.8>
11. Borg, M., Borg, M.: Requirements engineering and large language models: insights from a panel. *IEEE Softw.* **41**(2), 6–10 (2024). <https://doi.org/10.1109/MS.2023.3339934>
12. Siddeshwar, V., Alwidian, S., Makrehchi, M.: Goal model extraction from user stories using large language models, pp. 269–276 (2024). https://doi.org/10.1007/978-3-031-70245-7_19
13. Arulmohan, S., Meurs, M.J., Mosser, S.: Extracting domain models from textual requirements in the era of large language models. In: Proceedings - 2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion, Model 2023, pp. 580–587 (2023). <https://doi.org/10.1109/MODELS-C59198.2023.00096>
14. Feng, N., et al.: Normative requirements operationalization with large language models. In: Proceedings of the IEEE International Conference on Requirement Engineering, pp. 129–141 (2024). <https://doi.org/10.1109/RE59067.2024.00022>
15. Ruixiang, L., QiuJun, D., Xianhui, L., Chenglin, Z., Weidong, Z.: Requirement-service mapping technology in the industrial application field based on large language models. *Appl. Intell.* **55**(1) (2025). <https://doi.org/10.1007/s10489-024-05969-y>
16. Tikayat Ray, A., Cole, B.F., Pinon Fischer, O.J., Bhat, A.P., White, R.T., Mavris, D.N.: Agile methodology for the standardization of engineering requirements using large language models. *Systems* **11**(7) (2023). <https://doi.org/10.3390/systems11070352>
17. Ezzini, S., Abualhaija, S., Arora, C., Sabetzadeh, M.: Automated handling of anaphoric ambiguity in requirements: a multi-solution study. In: Proceedings - International Conference on Software Engineering, vol. 2022-May, pp. 187–199 (2022). <https://doi.org/10.1145/3510003.3510157>
18. Fantechi, A., Gnesi, S., Passaro, L., Semini, L.: Inconsistency detection in natural language requirements using ChatGPT: a preliminary evaluation. In: Proceedings of the IEEE International Conference on Requirements Engineering, vol. 2023-Septe, pp. 335–340 (2023). <https://doi.org/10.1109/RE57278.2023.00045>
19. Krishna, M., Gaur, B., Verma, A., Jalote, P.: Using LLMs in software requirements specifications: an empirical evaluation. In: Proceedings of the IEEE International Conference on Requirements Engineering, pp. 475–483 (2024). <https://doi.org/10.1109/RE59067.2024.00056>
20. Ren, S., Nakagawa, H., Tsuchiya, T.: Combining prompts with examples to enhance LLM-based requirement elicitation, pp. 1376–1381 (2024). <https://doi.org/10.1109/compsac61105.2024.00181>
21. Bertram, V., Kausch, H., Kusmenko, E., Ngiri, H., Rumpe, B., Venhoff, C.: Leveraging natural language processing for a consistency checking toolchain of automotive requirements. In: Proceedings of the IEEE International Conference on Requirements Engineering, vol. 2023-Septe, pp. 212–222 (2023). <https://doi.org/10.1109/RE57278.2023.00029>
22. Hasso, H., Fischer-Starcke, B., Geppert, H.: Quest-RE QUestion generation and exploration STrategy for requirements engineering. In: Proceedings - 32nd IEEE International Requirements Engineering Conference Workshops, REW 2024, pp. 3–11 (2024). <https://doi.org/10.1109/REW61692.2024.00006>
23. Luitel, D., Hassani, S., Sabetzadeh, M.: Improving requirements completeness: automated assistance through large language models. *Requir. Eng.* **29**(1), 73–95 (2024). <https://doi.org/10.1007/s00766-024-00416-3>

24. Uygun, Y., Momodu, V.: Local large language models to simplify requirement engineering documents in the automotive industry. *Prod. Manuf. Res.* **12**(1) (2024). <https://doi.org/10.1080/21693277.2024.2375296>
25. Yang, Y., Li, X., Ke, W., Liu, Z.: Automated prototype generation from formal requirements model. *IEEE Trans. Reliab.* **69**(2), 632–656 (2020). <https://doi.org/10.1109/TR.2019.2934348>
26. Norheim, J.J., Rebentisch, E.: Structuring natural language requirements with large language models. In: *Proc. - 32nd IEEE International Requirements Engineering Conference Workshops, REW 2024*, pp. 68–71 (2024). <https://doi.org/10.1109/REW61692.2024.00013>
27. Li, Y., Liu, P., Wang, H., Chu, J., Wong, W.E.: Evaluating large language models for software testing. *Comput. Stand. Interfaces* **93** (2025). <https://doi.org/10.1016/j.csi.2024.103942>
28. Jiao, Y., Wang, J., Li, R., Wang, F.-Y., Zhao, H., Xiong, G.: Intelligent generation of test cases for a parallel testing system: a case study on railway systems. *IEEE Intell. Transp. Syst. Mag.* **2**–13 (2024). <https://doi.org/10.1109/mits.2024.3435828>
29. Wang, C., Pastore, F., Goknil, A., Briand, L.C.: Automatic generation of acceptance test cases from use case specifications: an NLP-based approach. *IEEE Trans. Softw. Eng.* **48**(2), 585–616 (2022). <https://doi.org/10.1109/TSE.2020.2998503>
30. Dakhel, A.M., Nikanjam, A., Majdinasab, V., Khomh, F., Desmarais, M.C.: Effective test generation using pre-trained Large Language Models and mutation testing. *Inf. Softw. Technol.* **171** (2024). <https://doi.org/10.1016/j.infsof.2024.107468>
31. Ueda, K., Shikama, R., Shimizu, Y.: Automatic test cases generation with selection of training data for various system specifications. In: *Proceedings of the IEEE International Conference on Computer Communication and the Internet, ICCCI*, no. 2024, pp. 1–6 (2024). <https://doi.org/10.1109/ICCCI62159.2024.10674503>
32. Mai, P.X., Goknil, A., Shar, L.K., Pastore, F., Briand, L.C., Shaame, S.: Modeling security and privacy requirements: a use case-driven approach. *Inf. Softw. Technol.* **100**, 165–182 (2018). <https://doi.org/10.1016/j.infsof.2018.04.007>
33. Han, Y., Lee, D.H., Choi, B., Hinchey, M., In, H.P.: Value-driven V-model: from requirements analysis to acceptance testing. *IEICE Trans. Inf. Syst.* **E99D**(7), 1776–1785 (2016). <https://doi.org/10.1587/transinf.2015EDP7451>
34. Ferrari, A., Spagnolo, G.O., Gnesi, S.: PURE: a dataset of public requirements documents. In: *Proceedings - 2017 IEEE 25th International Requirement Engineering Conference, RE 2017*, pp. 502–505 (2017). <https://doi.org/10.1109/RE.2017.29>
35. ISO/IEC/IEEE15288. INTERNATIONAL STANDARD ISO/IEC/IEEE Systems and software engineering. *Int. Stand. ISO/IEC/IEEE 42010*, vol. First edit, pp. 1–104 (2018). <https://ieeexplore.ieee.org/servlet/opac?punumber=8559684>